

Relatório da Análise dos Algoritmos: Insertion-sort, Quick-sort and Merge-sort

Zaú Júlio A. Galvão

Recebido em 16 de novembro de 2020

1 Introdução

Um algoritmo é uma sequência de instruções finita, não ambíguas, com objetivo de gerar uma solução para um determinado problema[3]. Um dos problemas mais conhecidos a reorganização de elementos em vetor, já foi bem desenvolvido, tendo diversas soluções propostas. Neste estudo, analisei algumas destas soluções, especificamente o Insertion-sort, o Quick-sort e o Merge-sort, verificando características como tempo de execução e complexidade.

Para isto implementei computacionalmente as versões que julguei mais eficientes e dos algoritmos, utilizando a linguagem de programação C, por ser compilada e ter um excelente controle de memória [1], calculei o tempo médio de execução em alguns casos e verifiquei sua correspondência com o tempo esperado e sua complexidade assintótica.

2 Metodologia

A metodologia utilizada neste estudo pode ser descrita em 5 etapas de desenvolvimento linear, começando pela criação de uma sólida base teórica, a escolha e implementação de uma determinada versão do algoritmo, até o estudo de como avaliá-lo e mensurá-lo. Essa abordagem está descrita na Figura 1.

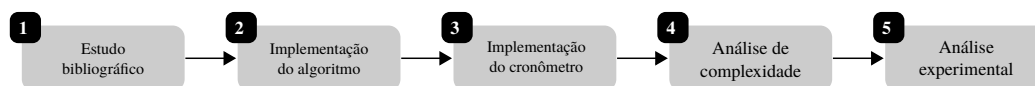


Figura 1: Ilustração da metodologia utilizada neste estudo. O processo foi dividido em 5 estágios: (1) estudo bibliográfico para fundamentar o desenvolvimento das implementações; (2) implementação computacional dos algoritmos Merge-sort, Insertion-sort e Quick-sort, utilizando a linguagem C; (3) elaboração do cronômetro para mensurar o tempo de execução; (4) análise da complexidade assintótica e do tempo de execução, no melhor, pior e médio caso. (5) análise experimental dos algoritmos estudados.

3 Fundamentação

3.1 Insertion-Sort

Um vetor com n elementos quaisquer é ordenado utilizando o método comparativo[3]. O algoritmo consiste em dividir o vetor em dois, um, a esquerda inicia unitário e como possui somente uma posição, já está ordenado, caracterizando o caso base. O segundo vetor, não ordenado, é iterado até encontrar um elemento superior que o elemento do outro vetor, em seguida a posição de inserção é escolhida e o elemento é ordenado. Este processo repete-se n^2 vezes até que o vetor esteja ordenado.

Algoritmo 1 (Insertion-sort). Caracteriza um algoritmo de ordenação de complexidade quadrática estável, ótimo para pequenos vetores de entrada.

algoritmo insertion-sort(A) { A : vetor}

```
1: para  $i$  de 1 até  $A.size - 1$  faça
2:    $k \leftarrow A[i]$ 
3:    $i \leftarrow j-1$ 
4:   enquanto  $i \geq 0$  e  $A[i] > k$  faça
5:      $A[i+1] \leftarrow A[i]$ 
6:      $i \leftarrow i - 1$ 
7:   fim enquanto
8: fim para
```

Tempo esperado:

$$T(n) = C1 * n + (C2 + C3) * (n - 1) + C4 * \sum_{j=1}^{n-1} (tj) + (C5 + C6) * \sum_{j=1}^{n-1} (tj) + C8 * (n - 1) \quad (1)$$

Melhor caso:

$$T_b(n) = C1 * n + (C2 + C3) * (n - 1) + C4 * (n - 1) + (C5 + C6) * (n - 2) + C8 * (n - 1) \quad (2)$$

$$T_b(n) = C * (n) \text{ ou } O(n) \quad (3)$$

Pior caso:

$$T_w(n) = C1 * n + (C2 + C3) * (n - 1) + C4 * (n - 1)(n) / 2 + (C5 + C6) * ((n - 1)(n) / 2 - 1) + C8 * (n - 1) \quad (4)$$

$$T_w(n) = C * (n^2) \text{ ou } O(n^2) \quad (5)$$

Caso médio:

$$T_a(n) = C1 * n + (C2 + C3) * (n - 1) + C4 / 2 * (n - 1)(n) / 2 + (C5 + C6) / 2 * ((n - 1)(n) / 2 - 1) + C8 * (n - 1) \quad (6)$$

$$T_a(n) = C * (n^2) \text{ ou } O(n^2) \quad (7)$$

Considera-se que o vetor A é indexado a partir de 1. A complexidade do algoritmo é da ordem de $O(n^2)$ em tempo e $O(1)$ em memória. $\square$

3.2 Merge-Sort

Esse algoritmo segue o princípio de dividir para conquistar, utilizando uma implementação recursiva, a cada chamada ele divide o vetor enviado como parâmetro em dois, até que o vetor recebido seja unitário, caso base onde já está ordenado, quando isto ocorre uma função mescla os dois vetores ordenando a ramificação do vetor original.

Algoritmo 2 (Merge-sort). Caracteriza um algoritmo de ordenação de complexidade linear, logarítmica, estável, com tempo de execução de todos os casos de ordem de $\theta(n \log n)$.

```
algoritmo merge-sort(A, s, e) {A: vetor, s: início do vetor, e:final do vetor}
    se  $s < e$  então
         $m \leftarrow (s + e) / 2$ 
        merge-sort(A, s, m)
        merge-sort(A, m + 1, e)
        merge(A, s, m)
    fim se
algoritmo merge(A, s, m, e) {A: vetor, s: início do vetor, m: meio do vetor, e:final do vetor}
     $i \leftarrow s$ 
     $j \leftarrow m + 1$ 
    para  $k$  de 1 até  $e - s + 1$  faça
        se  $A[i] < A[j]$  e  $i \leq m$  e  $j > e$  então
             $B[k] \leftarrow A[i]$ 
             $i \leftarrow i + 1$ 
        else
             $B[k] \leftarrow A[j]$ 
             $j \leftarrow j + 1$ 
        fim se
    fim para
    para  $k$  de 1 até  $e - s + 1$  faça
         $A[s + k] \leftarrow B[k]$ 
    fim para
```

Diferente dos demais algoritmos, o Merge-Sort tem o mesmo tempo para o melhor, pior e caso médio, podemos encontrá-lo a partir da seguinte equação de recorrência:

$$T(n) = 2 * T(n/2) + \Theta n \quad (8)$$

Logo, a complexidade de todos os casos corresponde a:

$$T(n) = \Theta(n * \log n) \quad (9)$$

Novamente, considera-se que o vetor A é indexado a partir de 1. A complexidade do algoritmo é da ordem de $\theta(n \log n)$ em tempo e $O(n)$ em memória, pois necessita de um vetor auxiliar de mesmo tamanho do vetor de entrada. $\square$

3.3 Quick-Sort

Esse algoritmo segue o princípio de dividir para conquistar, utilizando uma implementação recursiva, começamos do elemento mais à esquerda e buscamos o próximo valor menor ou igual ao elemento anterior, se encontrarmos um elemento menor, trocamos o elemento atual com o anterior. Caso contrário, ignoramos o elemento atual.

Algoritmo 3 (Quick-sort). Caracteriza também um algoritmo de ordenação de complexidade linear, logarítmica, instável, com tempo de execução do pior caso da ordem de n^2 .

algoritmo quick-sort(A, s, e) { A : vetor, s : início do vetor, e :final do vetor}

se $s < e$ **então**

$p \leftarrow \text{partition}(A, s, e)$

 quick-sort($A, s, p - 1$)

 quick-sort($A, p + 1, e$)

fim se

algoritmo partition(A, s, e) { A : vetor, s : início do vetor, e :final do vetor}

$k \leftarrow A[e]$

$i \leftarrow s - 1$

para j de s até $e - 1$ **faça**

se $A[j] < K$ **então**

$i \leftarrow i + 1$

$A[i] \leftrightarrow A[j]$

fim se

fim para

$A[i] \leftrightarrow A[e]$

retorne $i + 1$

Tempo esperado:

$$T(n) = T(k) + T(n * k - 1) + \Theta(n) \quad (10)$$

Melhor caso:

$$T_b(n) = 2 * T(n/2) + \Theta(n) \text{ ou } \Theta(n * n) \quad (11)$$

Pior caso:

$$T_w(n) = T(0) + T(n - 1) + \Theta(n) \text{ ou } T(n - 1) + \Theta(n) \quad (12)$$

Caso médio:

$$T_a(n) = T(n/9) + T(9n/10) + \Theta(n) \text{ ou } O(n * n) \quad (13)$$

Considera-se que o vetor A é indexado a partir de 1. A complexidade do algoritmo é da ordem de $O(n^2)$ em tempo e $O(n)$ em memória, pois necessita de um vetor auxiliar de mesmo tamanho do vetor de entrada. $\square$

99 4 Implementação dos Algoritmos

100 Para verificar se os tempos de execução estipulados no capítulo anterior estavam corretos, foi feita
101 a implementação de todos os algoritmos na linguagem de programação C. A escolha de cada versão,
102 de cada algoritmo, levou em consideração a melhor eficiência em relação a tempo de execução e uso
103 de memória.

104 4.1 Insertion-Sort

```
105 void insertionSort(int *A, int size);  
106  
107 void insertionSort(int *A, int size) {  
108     int i, j, k;  
109     for (j = 1; j < size; j++) {  
110         k = A[j];  
111         i = j - 1;  
112         while (i >= 0 && A[i] > k) {  
113             A[i + 1] = A[i];  
114             i--;  
115         }  
116         A[i + 1] = k;  
117     }  
118 }
```

119 4.2 Merge-Sort

```
120 void mergeSort(int A[], int B[], int s, int e);
121 void merge(int A[], int B[], int s, int m, int e);
122
123 void mergeSort(int A[], int B[], int s, int e) {
124     if (s < e) {
125         int m = (s + e) / 2;
126         mergeSort(A, B, s, m);
127         mergeSort(A, B, m + 1, e);
128         merge(A, B, s, m, e);
129     }
130 }
131
132 void merge(int A[], int B[], int s, int m, int e) {
133     int i = s;
134     int j = m + 1;
135     for (int k = 0; k < (e - s + 1); k++) {
136         if ((A[i] < A[j] && i <= m) || j > e) {
137             B[k] = A[i];
138             i++;
139         } else {
140             B[k] = A[j];
141             j++;
142         }
143     }
144     for (int k = 0; k < (e - s + 1); k++) {
145         A[s + k] = B[k];
146     }
147 }
```

4.3 Quick-Sort

```
void quickSort(int *A, int s, int e);
int partition(int *A, int s, int e);
void swap(int *a, int *b);

void quickSort(int *A, int s, int e) {
    if (s < e) {
        int p = partition(A, s, e);
        quickSort(A, s, p - 1);
        quickSort(A, p + 1, e);
    }
}

int partition(int *A, int s, int e) {
    int k = A[e];
    int i = s - 1;
    for (int j = s; j < e; j++) {
        if (A[j] < k) {
            i++;
            swap(&A[i], &A[j]);
        }
    }
    swap(&A[i + 1], &A[e]);
    return i + 1;
}

void swap(int *a, int *b) {
    int t;

    t = *b;
    *b = *a;
    *a = t;
}
```

5 Experimentos

Após a implementação dos algoritmos a prioridade se tornou encontrar a melhor forma de mensurar o tempo de execução dos experimentos. Este processo levou em consideração a premissa de que qualquer um deverá poder replicar estes experimentos sem nenhuma complicação ou inconsistência. A primeira opção utilizava `CLOCK_MONOTONIC`, uma função da linguagem C, nativa e funcional somente em sistemas UNIX. Esta função possuía uma alta precisão, na ordem de nanosegundos, pois utiliza o `clock_id` da CPU para mensurar o tempo que se passou desde o início do sistema operacional. Contudo, para manter a universalidade para fácil replicação destes experimentos, utilizei outra biblioteca que opera em sistemas UNIX e POSIX, `gettimeofday`, da biblioteca `sys/time.h`, com precisão de microssegundos [2].

Além da função de cronômetro, também implementei funções de ordenação, ordenação reversa e preenchimento aleatório do vetor, para testar os algoritmos em diferentes casos. Os experimentos foram executados em um computador 64 bits, com sistema operacional UNIX, com um kernel mínimo

194 compilado exclusivamente para executar somente o necessário e estes experimentos. Após a inicia-
195 lização total do sistema operacional, o computador executou o script 100x, logo em seguida, na 101ª
196 vez, coletou os resultados.

197 5.1 Utilitários do experimento

```
198 #include <stdio.h>
199 #include <stdlib.h>
200 #include <string.h>
201 #include <sys/time.h>
202
203 #include "src/insertion-sort.c"
204 #include "src/merge-sort.c"
205 #include "src/quick-sort.c"
206
207 void shuffle(int *A, int size);
208 void decreasing(int *A, int size);
209 int *allocate(int size);
210 int *copyArray(int *A, int size);
211 double orderArray(int *A, char *alg, char sourceOrder, int size);
212 void checkUsage(int argc);
213
214 void shuffle(int *A, int size) {
215     struct timeval tv;
216     gettimeofday(&tv, NULL);
217     int usec = tv.tv_usec;
218     srand48(usec);
219
220     if (size > 1) {
221         for (int i = size - 1; i > 0; i--) {
222             int j = (unsigned int)(drand48() * (i + 1));
223             swap(&A[i], &A[j]);
224         }
225     }
226 }
227
228 void decreasing(int *A, int size) {
229     for (int i = size, j = 0; i > 0; i--, j++) {
230         A[j] = i;
231     }
232 }
233
234 int *copyArray(int *A, int size) {
235     int *B;
236     B = (int *)malloc(size * sizeof(int));
237     for (int i = 0; i < size; i++) {
238         B[i] = A[i];
239     }
240     return B;

```

```
241     }
242
243     int *allocate(int size) {
244         int *A;
245         A = (int *)malloc(size * sizeof(int));
246         for (int i = 0; i < size; i++) {
247             A[i] = i;
248         }
249         return A;
250     }
251
252     double orderArray(int *A, char *alg, char sourceOrder, int size) {
253         struct timeval begin, end;
254
255         if (sourceOrder == 'r') {
256             shuffle(A, size);
257         }
258         if (sourceOrder == 'd') {
259             decreasing(A, size);
260         }
261
262         if (strcmp(alg, "mergeSort") == 0) {
263             int *B = copyArray(A, size);
264             gettimeofday(&begin, 0);
265             mergeSort(A, B, 0, size - 1);
266             gettimeofday(&end, 0);
267
268             free(B);
269         }
270         if (strcmp(alg, "quickSort") == 0) {
271             gettimeofday(&begin, 0);
272             quickSort(A, 0, size - 1);
273             gettimeofday(&end, 0);
274         }
275         if (strcmp(alg, "insertionSort") == 0) {
276             gettimeofday(&begin, 0);
277             insertionSort(A, size);
278             gettimeofday(&end, 0);
279         }
280
281         long seconds = end.tv_sec - begin.tv_sec;
282         long microseconds = end.tv_usec - begin.tv_usec;
283         return seconds + microseconds*1e-6;
284     }
285
286     int main(int argc, char *argv[]) {
287         /* */
288         checkUsage(argc);
289     }
```

```
290     int range = atoi(argv[1]);
291     int maxSize = atoi(argv[2]) + range;
292     char order = argv[3][0];
293     char *alg = argv[4];
294     int meanSize = atoi(argv[5]);
295
296     double mean;
297     int size = range;
298
299     while (size < maxSize) {
300         mean = 0;
301         for (int i = 0; i < meanSize; i++) {
302             int *A = allocate(size);
303             double time_taken = orderArray(A, alg, order, size);
304             free(A);
305             mean += time_taken;
306         }
307         printf("%f_%d\n", (double)mean/meanSize, size);
308         size += range;
309     }
310
311     return 0;
312 }
```

5.2 Resultados

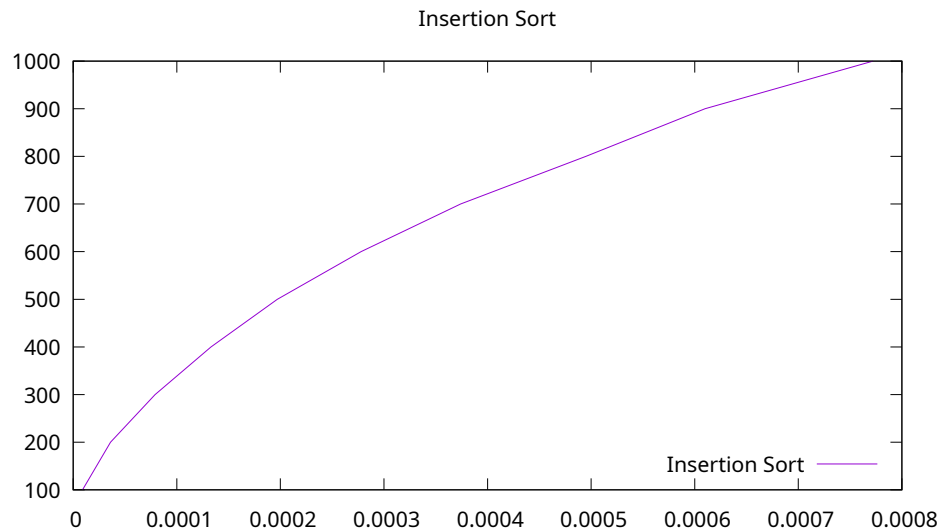


Figura 2: Insertion Sort: $O(n^2)$. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

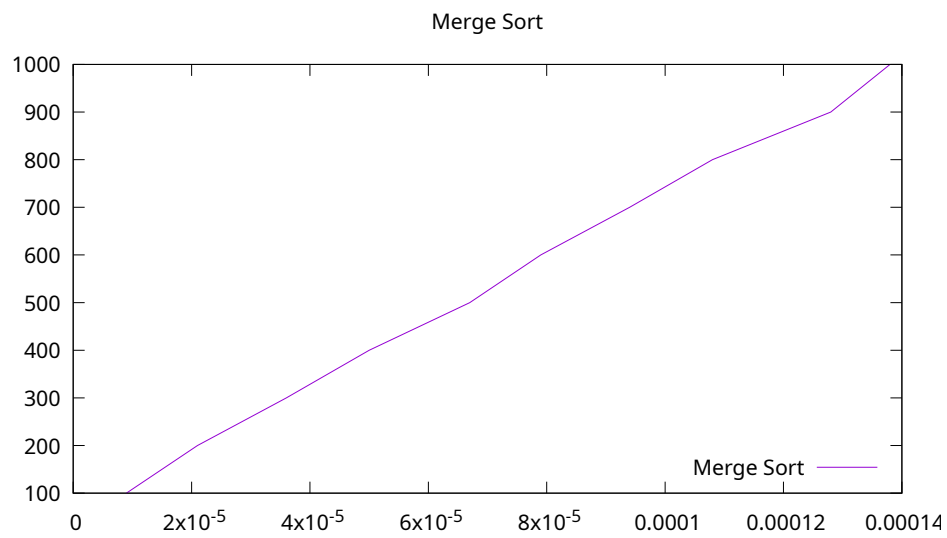


Figura 3: Merge Sort: $\theta(n \log n)$. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

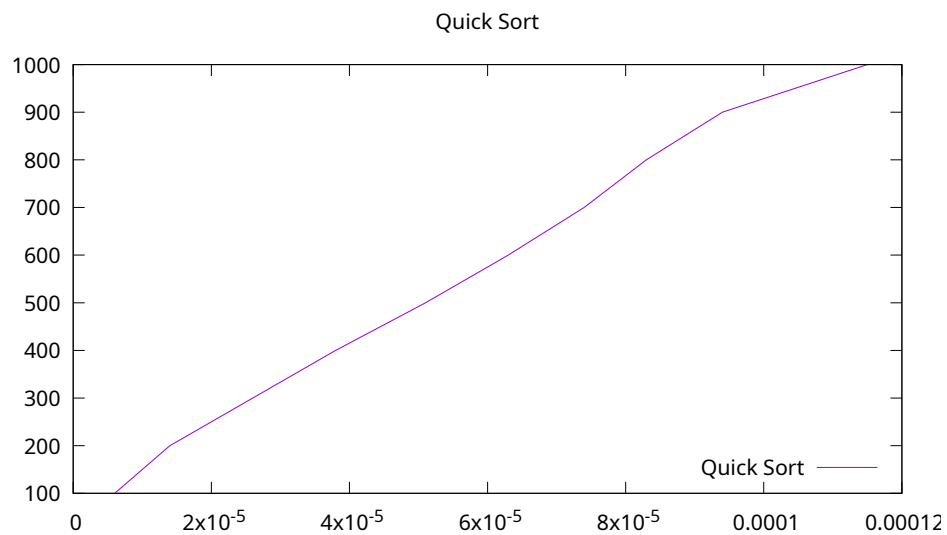


Figura 4: Quick Sort: $O(n^2)$. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

314 5.3 Comparativo Geral

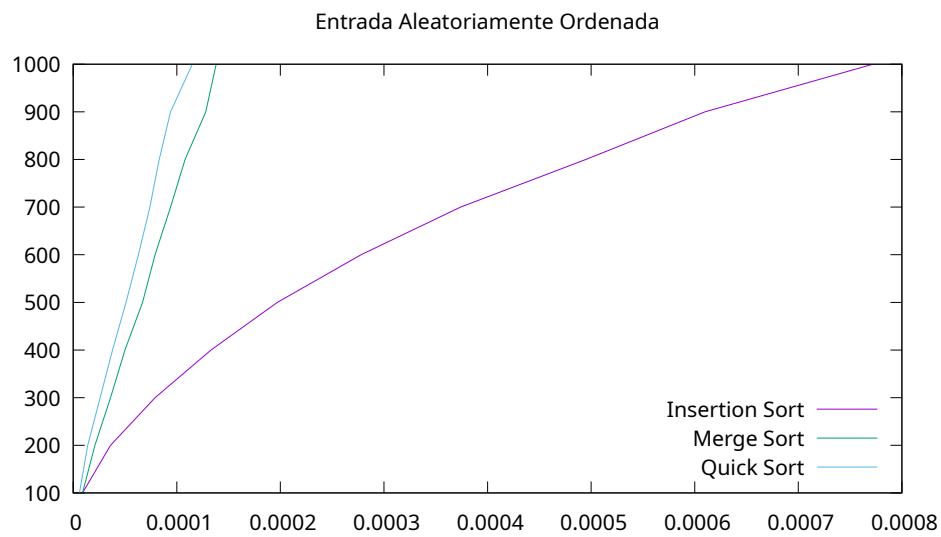


Figura 5: Comparativo Geral. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

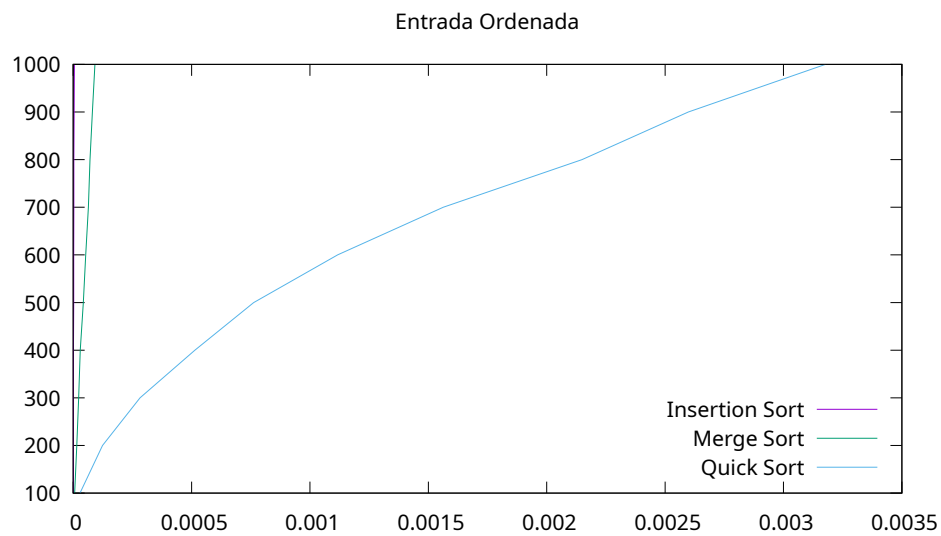


Figura 6: Comparativo Geral: Entrada de Vetor Pré-Ordenado. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

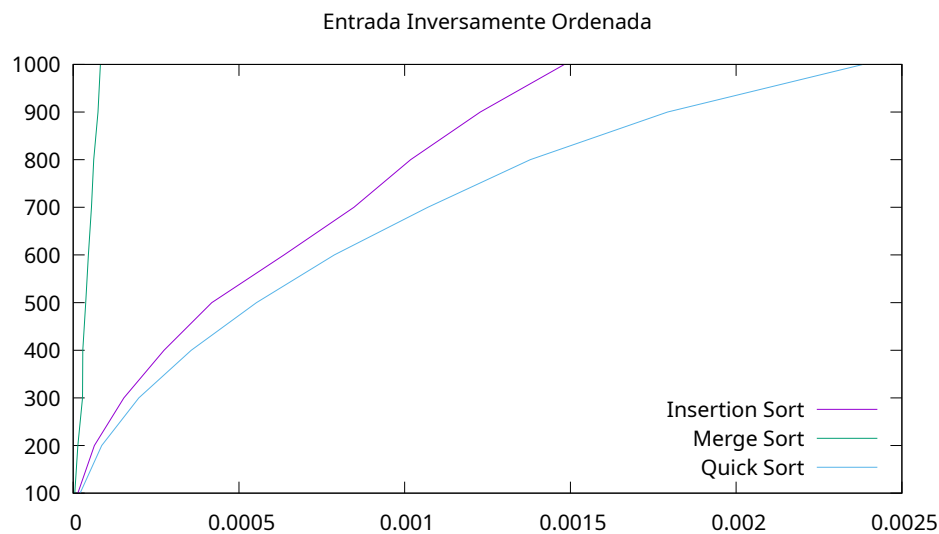


Figura 7: Comparativo Geral: Entrada de Vetor Pré-Ordenado Inversamente. Eixo X em microssegundos, eixo Y corresponde ao tamanho do vetor.

6 Conclusão

Com este estudo podemos concluir que o algoritmo Insertion Sort é o pior em tempo de execução entre os demais, contudo é o único com custo de memória igual a 1. O algoritmo mais eficiente dentre os analisados é definitivamente o Merge Sort, por possuir menor complexidade e tempo de execução, comprovado experimentalmente, inferior aos demais. Além de ser o melhor em tempo de execução, é estável e facilmente paralelizável.

Referências

- [1] Brian W Kernighan and Dennis M Ritchie. *The C programming language*. 2006.
- [2] Sandra Loosemore, Richard M Stallman, Andrew Oram, and Roland McGrath. *The gnu c library reference manual*. 1993.
- [3] Marco Medina and Cristina Ferting. *Algoritmos e programação: teoria e prática*. Novatec Editora, 2006.